

Model-Based Testing with QuickCheck

Arnaud Bailly - @dr_c0d3

2024-03-15

Agenda

- ▶ Why?

Agenda

- ▶ Why?
- ▶ Property-Based Testing

Agenda

- ▶ Why?
- ▶ Property-Based Testing
- ▶ quickcheck-dynamic: Practice

Agenda

- ▶ Why?
- ▶ Property-Based Testing
- ▶ quickcheck-dynamic: Practice
- ▶ quickcheck-dynamic: Theory

Agenda

- ▶ Why?
- ▶ Property-Based Testing
- ▶ quickcheck-dynamic: Practice
- ▶ quickcheck-dynamic: Theory
- ▶ Conclusion

Why?

The Quest for Better Software

Going full circle

Automata Testing → Test-Driven Development → Property-Based Testing → Model-Based Testing

Engineering vs. Science

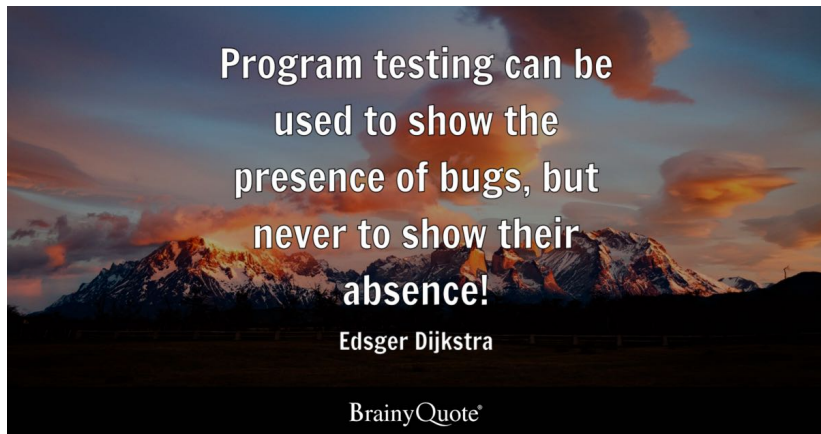


Figure 1: Dijkstra on Testing

My Goals for this Talk

- ▶ Share my experience with Model-Based testing to...

My Goals for this Talk

- ▶ Share my experience with Model-Based testing to...
- ▶ ... spark interest in the use of this family of tools and ...

My Goals for this Talk

- ▶ Share my experience with Model-Based testing to...
- ▶ ... spark interest in the use of this family of tools and ...
- ▶ ... trigger contributions to improve quickcheck-dynamic

What can you expect?

- ▶ An overview of an interesting new tool

What can you expect?

- ▶ An overview of an interesting new tool
- ▶ “From the trenches” report on the usefulness of model-based testing

What can you expect?

- ▶ An overview of an interesting new tool
- ▶ “From the trenches” report on the usefulness of model-based testing
- ▶ Tips & tricks on how and what to model

Property-Based Testing

What is Property-Based Testing?

- ▶ Example-based testing $\rightarrow$ Property-Based Testing

What is Property-Based Testing?

- ▶ Example-based testing → Property-Based Testing
- ▶ Generalise examples to express *properties* of the SUT

Classical examples

```
elements_are_sorted :: [a] -> Property
elements_are_sorted list =
  let sorted = sort list
  in undefined    --- TBD
```

Classical examples

```
roundtrip_encoding_decoding :: a -> Property
roundtrip_encoding_decoding a =
  let encoded = encode a
      decoded = decode encoded
  in decoded === Just a
```

QuickCheck Core Features

- ▶ Generate some number of arbitrary values. . .

QuickCheck Core Features

- ▶ Generate some number of arbitrary values. . .
- ▶ . . . when property fails, shrink the failing input . . .

QuickCheck Core Features

- ▶ Generate some number of arbitrary values. . .
- ▶ . . . when property fails, shrink the failing input . . .
- ▶ . . . to provide smallest counterexample upon failure

Additional Key Features

- ▶ cover expected distribution of test cases

Additional Key Features

- ▶ cover expected distribution of test cases
- ▶ Build generators combining smaller pieces

Additional Key Features

- ▶ cover expected distribution of test cases
- ▶ Build generators combining smaller pieces
- ▶ Derive generators generically

Additional Key Features

- ▶ cover expected distribution of test cases
- ▶ Build generators combining smaller pieces
- ▶ Derive generators generically
- ▶ ... and much more :right_pointing_hand: QuickCheck on hackage

Stateful QuickCheck

- ▶ Monadic version of QC

Stateful QuickCheck

- ▶ Monadic version of QC
- ▶ Running properties against stateful (IO) programs

Testing Monadic Code with QuickCheck

Koen Claessen

John Hughes

Department of Computer Science, Chalmers University of Technology

Abstract

QuickCheck is a previously published random testing tool for Haskell programs. In this paper we show how to use it for testing monadic code, and in particular imperative code written using the *ST* monad. QuickCheck tests a program against a specification: we show that QuickCheck's specification language is sufficiently powerful to represent common forms of specifications: algebraic, model-based (both functional and relational), and pre-/post-conditional. Moreover, all these forms of specification can be used directly for testing. We define a new language of monadic properties, and make a link between program testing and the notion of observational equivalence.

and postconditions. We introduce a new *monadic property language*, which simplifies these specifications considerably. Our main examples are an imperative implementation of queues, and the Union/Find algorithm.

The structure of the paper is as follows. In the next section, we introduce QuickCheck as previously described. In section 3 we introduce the queue example. Section 4 presents an algebraic specification of imperative queues, and shows how it can be used for testing, introducing the key idea of testing observational equivalence. The next section repeats the exercise for a model-based specification instead. Section 6 draws lessons from the queue example, in particular the desirability of extending QuickCheck with a language of monadic properties. The next three sections introduce this language, present a formal semantics, and sketch its implementation.

Figure 2: Testing Monadic code with QuickCheck

Stateful QuickCheck

- ▶ Define a *Domain-Specific Language* of inputs to the SUT

Stateful QuickCheck

- ▶ Define a *Domain-Specific Language* of inputs to the SUT
- ▶ Provide a *Model* of the expected behaviour

Stateful QuickCheck

- ▶ Define a *Domain-Specific Language* of inputs to the SUT
- ▶ Provide a *Model* of the expected behaviour
- ▶ Run the *Model* and the SUT in parallel with an *arbitrary* sequence of actions

Stateful QuickCheck

- ▶ Define a *Domain-Specific Language* of inputs to the SUT
- ▶ Provide a *Model* of the expected behaviour
- ▶ Run the *Model* and the SUT in parallel with an *arbitrary* sequence of actions
- ▶ Compare the results

Example: A Basic Repository Interface

Real-life example drawn from personal tool that needs to read/write events in a database.

Define a GADT of “interesting” Actions to run

```
-- / Relevant commands issued to the underlying DB
data Action a where
  WriteEvent :: Event -> Action ()
  ReadEvents :: Pagination -> Action EventsQueryResult
  ...
  NewUser :: UserProfile -> Action ()
  SwitchUser :: Text -> Action ()
```

Interpret actions against a Model

```
interpret :: (Monad m, Eq a, Show a) => Action a -> StateT
interpret (WriteEvent f) = do
    modify $ \m@Model {events} -> m {events = events |> Event f
    pure $ Just ()
interpret (ReadEvents (Page pageNum size)) = do
    es <- getEvents
    ...
```

Generate (meaningful) sequence of Actions:

```
generateActions :: Model -> Gen Actions
```

```
generateActions model =
```

```
  Actions <$>
```

```
    (arbitrary >>=
```

```
      generateAction startTime model . getPositive)
```

Run interpreter and SUT in parallel and check conformance:

```
canReadFlowsAndTracesWritten ::  
  (DB db, HasCallStack) =>  
  FilePath -> (forall x. db x -> IO x) -> Property  
canReadFlowsAndTracesWritten dbFile nt =  
  forAllShrink (generateActions start) shrinkActions $  
    \ (Actions actions) -> monadicIO $ do  
      res <- run $ nt $  
        initLogStorage >> evalStateT (validateActions actions) $  
        forM_ res monitorErrors  
      assert $ all isNothing res
```

quickcheck-dynamic: Practice

The need for a better tool

- ▶ Ad-hoc model-based testing has limitations

The need for a better tool

- ▶ Ad-hoc model-based testing has limitations
- ▶ Quviq people are experts in these techniques and they were building a tool for IOG

The need for a better tool

- ▶ Ad-hoc model-based testing has limitations
- ▶ Quviq people are experts in these techniques and they were building a tool for IOG
- ▶ I was involved in a complex project that would benefit from a comprehensive testing strategy

Hydra - Overview

- ▶ A Layer-2 network for UTXO blockchains based on *State channels*

Hydra - Overview

- ▶ A Layer-2 network for UTXO blockchains based on *State channels*
- ▶ Complex on-chain protocol advancing a state-machine through transactions

Hydra - Overview

- ▶ A Layer-2 network for UTxO blockchains based on *State channels*
- ▶ Complex on-chain protocol advancing a state-machine through transactions
- ▶ Off-chain leader-based distributed consensus

Hydra - Overview

- ▶ A Layer-2 network for UTxO blockchains based on *State channels*
- ▶ Complex on-chain protocol advancing a state-machine through transactions
- ▶ Off-chain leader-based distributed consensus
- ▶ More details at <https://hydra.family>

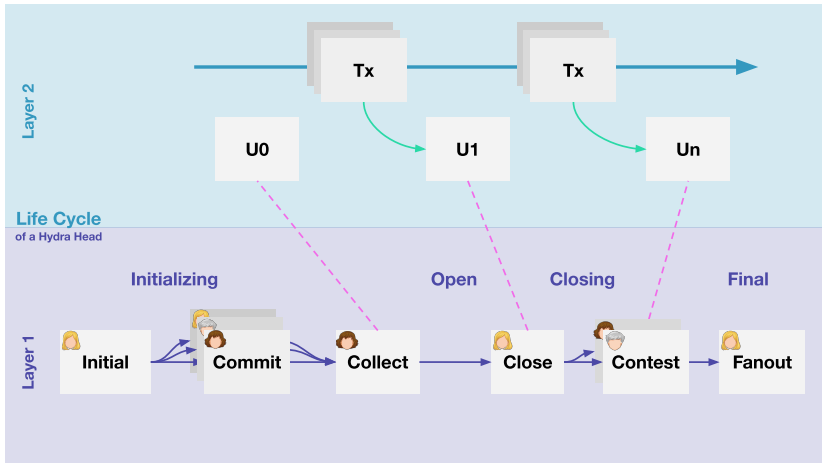


Figure 3: High-level Hydra Protocol

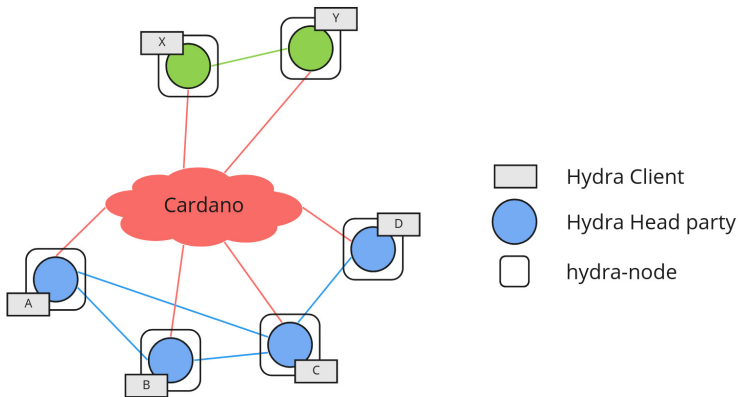


Figure 4: Hydra Deployment

Hydra - Model

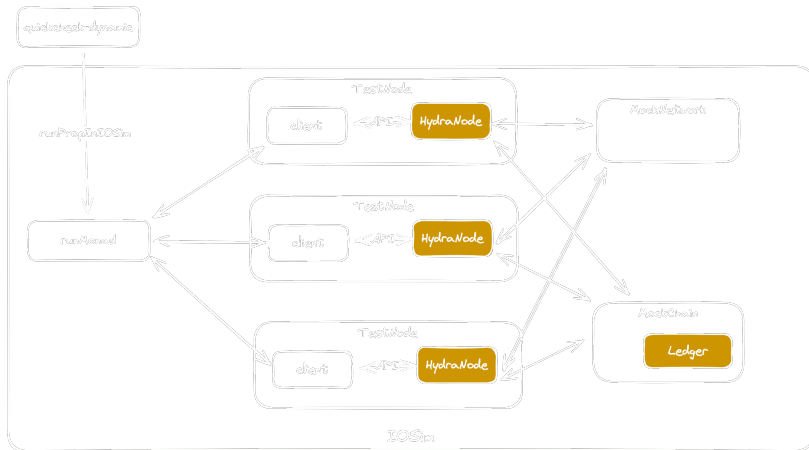


Figure 5: Hydra Test Architecture

Hydra - Model

- ▶ Simulation-based testing + MBT = Powerful *combo*
(FoundationDB, Quviq's PULSE)

Hydra - Model

- ▶ Simulation-based testing + MBT = Powerful *combo* (FoundationDB, Quviq's PULSE)
- ▶ Runs network of nodes using io-sim, a Free-monad based framework for simulating Haskell runtime

Hydra - Model

- ▶ Simulation-based testing + MBT = Powerful *combo* (FoundationDB, Quviq's PULSE)
- ▶ Runs network of nodes using io-sim, a Free-monad based framework for simulating Haskell runtime
- ▶ See Philip Kant's presentation at BobKonf 2022

Hydra - Properties

Original research paper defines several key properties

CONFLICT-FREE LIVENESS (HEAD): In presence of a network adversary, a conflict-free execution satisfies the following condition: For any transaction tx input via $(\mathbf{new}, tx)$, $tx \in \bigcap_{i \in [n]} \overline{C}_i$ eventually holds.

Figure 6: Conflict-free liveness

Properties are (manually) expressed as *Dynamic Logic* formulas

```
conflictFreeLiveness = do
  anyActions_
  getModelStateDL >>= \case
    Open{} -> do
      payment <- forAllNonVariableQ (nonConflictingTx st)
      tx <- action $ Model.NewTx payment
      eventually (ObserveConfirmedTx tx)
```

```
Hydra.Model
check conflict-free liveness
  ++ OK, passed 100 tests.

Action polarity (966 in total):
100.0% +

Actions (966 in total):
34.5% +NewTx
17.4% +Commit
10.4% +StopTheWorld
9.7% +Seed
8.5% +Init
5.6% +RollbackAndForward
3.1% +Abort
2.9% +Close
2.8% +Fanout
2.1% +ObserveConfirmedTx
2.1% +Wait

Transitions (966 in total):
45.2% Open -> Open
13.9% Initial -> Initial
9.7% Start -> Idle
9.5% Idle -> Initial
5.9% Final -> Final
5.0% Initial -> Open
3.1% Initial -> Final
2.9% Open -> Closed
2.8% Closed -> Final
1.1% Closed -> Closed
0.6% Start -> Start
0.2% Idle -> Idle

Finished in 4.5131 seconds
1 example, 0 failures
Test suite tests: PASS
```

Generate "adversarial" actions



Figure 7: Hydra Property Execution

- ▶ Fast finality protocol for Cardano

Peras

- ▶ Fast finality protocol for Cardano
- ▶ Early work integrating research, formal methods, engineering. . .

Peras

- ▶ Fast finality protocol for Cardano
- ▶ Early work integrating research, formal methods, engineering. . .
- ▶ Use quickcheck-dynamic to produce *Executable Specification* from *Formal Specification*

The slogan is:

Agda Proofs are Quickcheck Tests

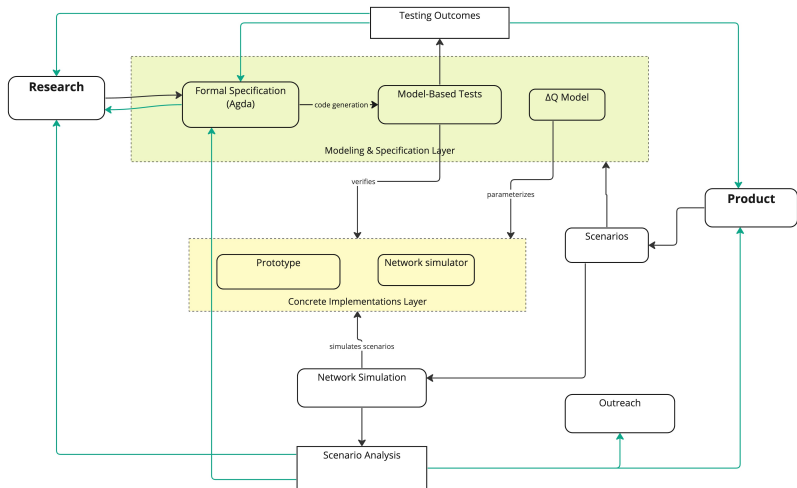


Figure 8: Development Workflow

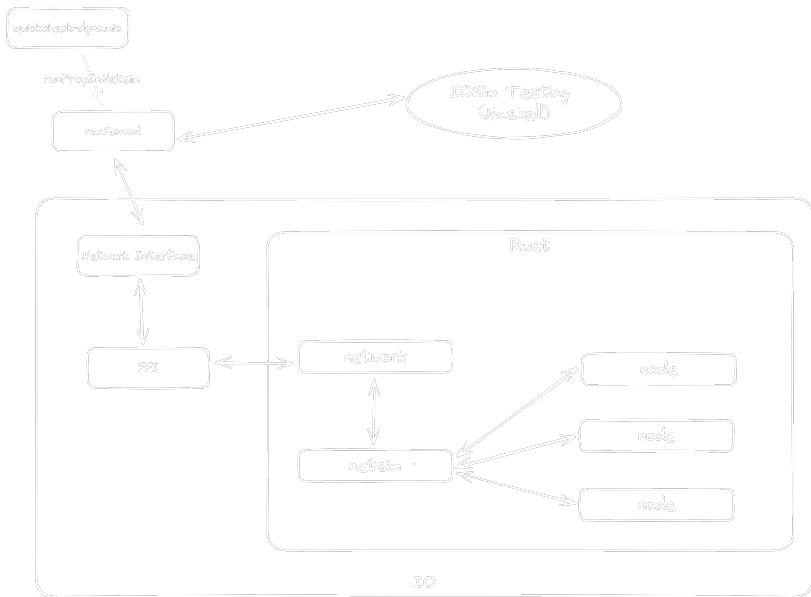


Figure 9: Testing Architecture

Testing *Common Prefix* property

```
describe "IOSim Network" $  
  prop "Chain progress" $  
    prop_common_prefix iOSimNetwork
```

```
describe "Netsim Network" $  
  prop "Chain progress" $  
    withMaxSuccess 20 $  
      prop_common_prefix netsimNetwork
```

Testing *Common Prefix* property

```
chainCommonPrefix = do
  anyActions_
  getModelStateDL >>= \Network{nodeIds} -> do
    anyAction
    chains <- forM nodeIds (action . ObserveBestChain)
    void $ action $ ChainsHaveCommonPrefix chains
```


quickcheck-dynamic: Theory

What is it?

- ▶ A library for *Property-Based* testing of *stateful* systems

What is it?

- ▶ A library for *Property-Based* testing of *stateful* systems
- ▶ Developed by Quviq while working at Input Output on Cardano

What is it?

- ▶ A library for *Property-Based* testing of *stateful* systems
- ▶ Developed by Quviq while working at Input Output on Cardano
- ▶ Open-sourced in 2022

Principles

- ▶ Model programs as *labelled transition systems*

Principles

- ▶ Model programs as *labelled transition systems*
- ▶ Express properties as Dynamic Logic formulas

Principles

- ▶ Model programs as *labelled transition systems*
- ▶ Express properties as Dynamic Logic formulas
- ▶ Generate sequence of actions respecting the property

Principles

- ▶ Model programs as *labelled transition systems*
- ▶ Express properties as Dynamic Logic formulas
- ▶ Generate sequence of actions respecting the property
- ▶ Run sequence of actions against SUT and find bugs!

Model - Basic steps

Specify possibles *actions* and `initialState`

```
instance StateModel KV where
  data Action KV a where
    Put  :: String -> Int -> MySystem ()
    Get  :: String -> MySystem (Maybe Int)

  initialState :: KV
  initialState = mempty
```

Model - Basic steps

Generate actions according to the current state

```
instance StateModel KV where
  initialState = ...
  arbitraryAction env state =
    oneof [ genWrite
            , genRead
          ]
    ...
```

Model - Basic steps

Define nextState transition

```
instance StateModel KV where
  initialState = ...
  arbitraryAction = ...
  nextState state (Put k v) variable =
    insert state k v
  nextState state (Get k) variable =
    state
```

Implementation

Relate the specification to an actual (monadic) implementation

```
instance RunModel KV StoreM where
  perform state (Put k v) env = do { putStore k v }
  perform state (Get k) env = do { getStore k }
```

Implementation

Specify postconditions that should hold after each Action

```
instance RunModel KV StoreM where
  perform state (Put k v) env = ...
  postcondition (before, after) (Put k) env v = pure True
  postcondition (before, after) (Get k) env v =
    pure $ v == lookup env k
```

Dynamic Logic

- ▶ A *Modal logic* to define properties over *traces* of a system

Dynamic Logic

- ▶ A *Modal logic* to define properties over *traces* of a system
- ▶ Exposed as both an *expression* and *monadic* DSL

Dynamic Logic

- ▶ A *Modal logic* to define properties over *traces* of a system
- ▶ Exposed as both an *expression* and *monadic* DSL
- ▶ Provide combinators to relate results of actions to predicates over the state of the SUT

Foundations of Computing Series



David Harel
Dexter Kozen
Jerzy Tiuryn

DYNAMIC LOGIC

Dynamic Logic - Syntax

Modality: $[a]p$

- After a p

Dynamic Logic - Syntax

Modality: $[a]p$

- ▶ After a p
After action a , p holds

Dynamic Logic - Syntax

Modality: $[a]p$

- ▶ After a p
After action a , p holds
- ▶ AfterAny p

Dynamic Logic - Syntax

Modality: $[a]p$

- ▶ $\text{After } a \text{ } p$
After action a , p holds
- ▶ $\text{AfterAny } p$
After any action a , p holds

Constants: $\bot$, $\emptyset$

► Stop

Constants: $\bot$, $\emptyset$

► Stop

When execution Stops, expression is true

Constants: $\bot$, $\emptyset$

- ▶ Stop
When execution Stops, expression is true
- ▶ Empty

Constants: $\bot$, $\emptyset$

- ▶ Stop
When execution Stops, expression is true
- ▶ Empty
When no execution is possible, expression is false

Alternatives: $[a \cup b]p$, $[a \cap b]p$

► Alt Demonic $f \ g$

Alternatives: $[a \cup b]p$, $[a \cap b]p$

- ▶ Alt Demonic $f \ g$
After a or b p must hold

Alternatives: $[a \cup b]p$, $[a \cap b]p$

- ▶ Alt Demonic $f \ g$
After a or b p must hold
- ▶ Alt Angelic $f \ g$

Alternatives: $[a \cup b]p$, $[a \cap b]p$

- ▶ Alt Demonic $f \ g$
After a or b p must hold
- ▶ Alt Angelic $f \ g$
 p must hold After either a or b

Dynamic Logic - Monadic Syntax

Provide a more convenient syntax

```
newtype DL s a = DL { ... }  
    deriving (Functor, Applicative, Alternative, Monad)  
  
action :: Action s a -> DL s (Var a)  
anyAction :: DL s ()  
anyActions :: Int -> DL s ()  
getModelStateDL :: DL s s  
...
```

Combinator to Generate arbitrary values within expression

```
class Quantifiable q where
  type Quantifies q

  quantify :: q -> Quantification (Quantifies q)

forallQ :: Quantifiable q => q -> DL s (Quantifies q)
```

Examples

From Thread Registry example

Can register an unbound thread under a new name

```
canRegister = do
  anyActions_
  name <- pickFreshName
  tid <- pickAlive
  unregisterNameAndTid name tid
  action $ Register name tid
```


Cannot register an already registered thread under a new name

```
canRegisterNoUnregister = do
  anyActions_
  name <- pickFreshName
  tid <- pickAlive
  action $ Register name tid
pure ()
```

Running Properties

Tie *Dynamic Logic* expression and *Actions* execution into a Property

```
forallDL ::  
    (DL.DynLogicModel s, Testable a)  
=> DL s ()  
-> (Actions s -> a)  
-> Property
```

Running Properties

Tie *StateModel* and *RunModel*, interpreting Actions against the SUT

```
runActions
```

```
  :: forall state m e
  . ( StateModel state
    , RunModel state m
    )
=> Actions state
-> PropertyM m (Annotated state, Env m)
```

Shrinking

Sequence of actions that fail are *shrunk* while respecting DL expression

- ▶ Shorten `anyActions_` traces

Shrinking

Sequence of actions that fail are *shrank* while respecting DL expression

- ▶ Shorten `anyActions_` traces
- ▶ Shrink `action` and `anyAction` data according to model's `shrinkAction`

Shrinking

Sequence of actions that fail are *shrank* while respecting DL expression

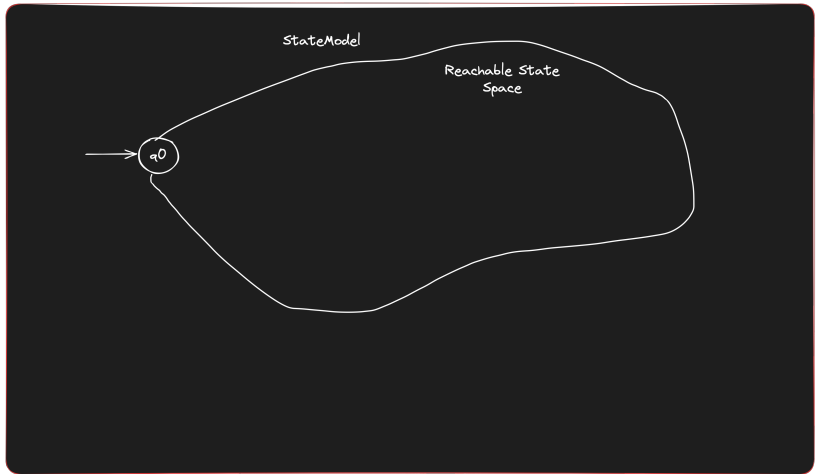
- ▶ Shorten `anyActions_` traces
- ▶ Shrink action and `anyAction` data according to model's `shrinkAction`
- ▶ Shrink Quantifiable values generated

Shrinking

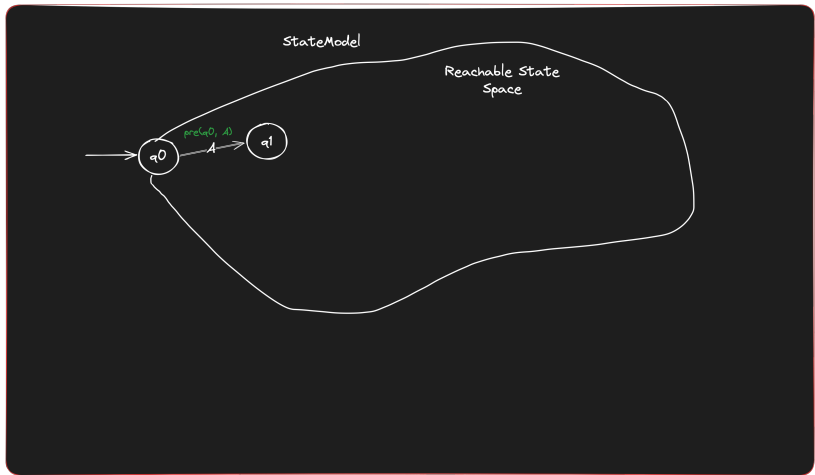
Sequence of actions that fail are *shrank* while respecting DL expression

- ▶ Shorten `anyActions_` traces
- ▶ Shrink action and `anyAction` data according to model's `shrinkAction`
- ▶ Shrink Quantifiable values generated
- ▶ precondition filters invalid sequence of actions

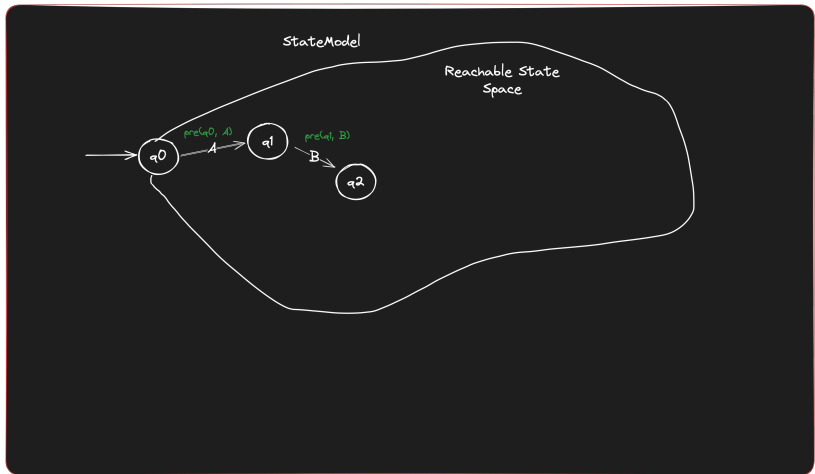
Anatomy of a test run



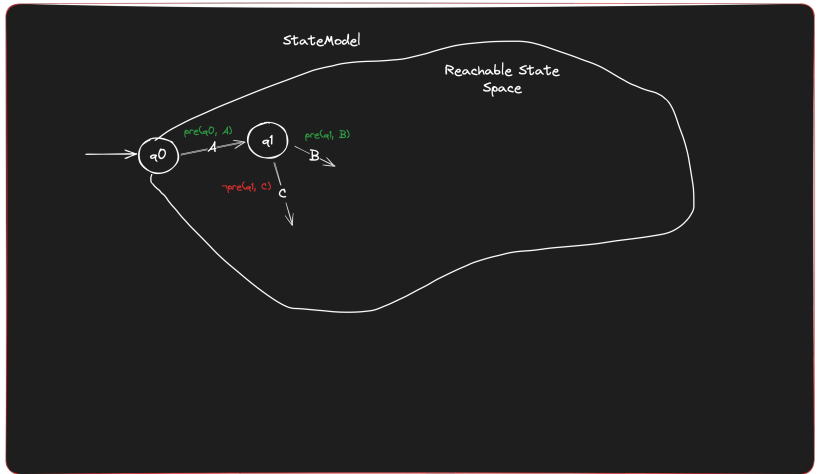
Anatomy of a test run



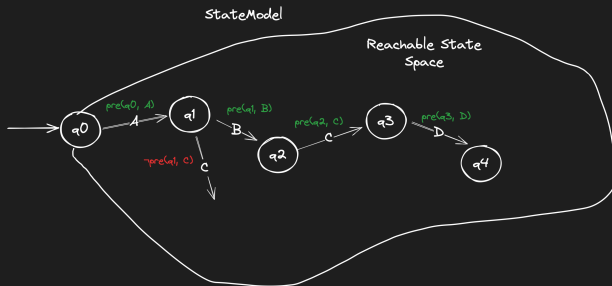
Anatomy of a test run



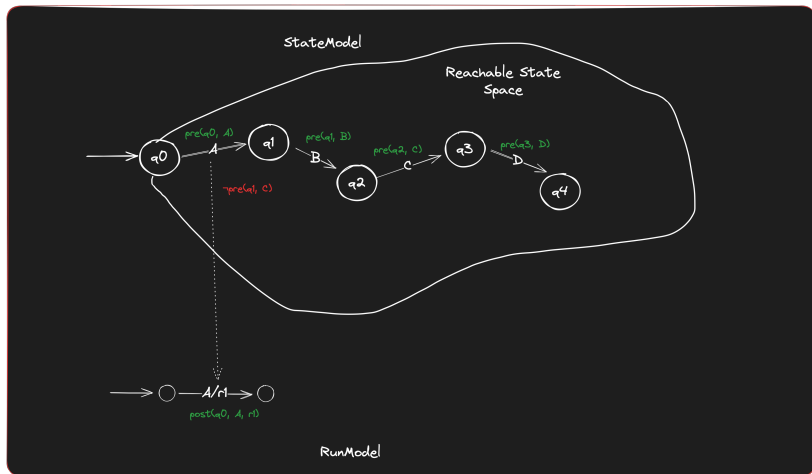
Anatomy of a test run



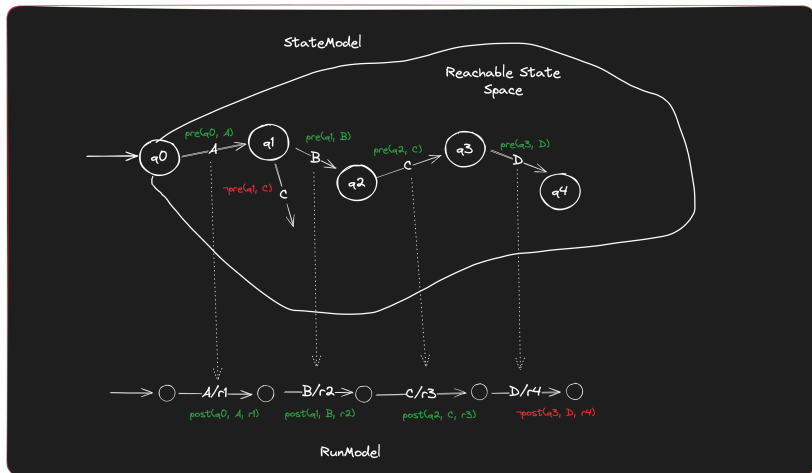
Anatomy of a test run



Anatomy of a test run



Anatomy of a test run



Testing for safety

- ▶ We often want to test that *nothing bad can happen*, eg. safety properties

Testing for safety

- ▶ We often want to test that *nothing bad can happen*, eg. safety properties
- ▶ We can express that in the model through *Polarity* of Actions:

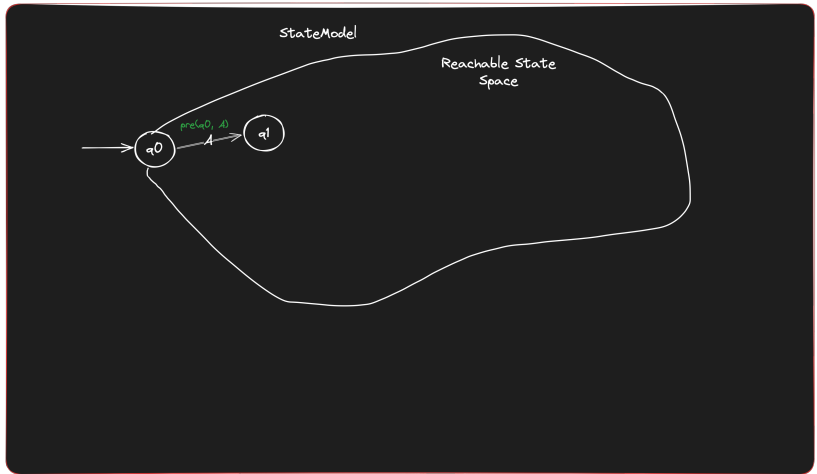
Testing for safety

- ▶ We often want to test that *nothing bad can happen*, eg. safety properties
- ▶ We can express that in the model through *Polarity* of Actions:
 - ▶ A *Positive* action is valid w.r.t to the state and is expected to *succeed* when run

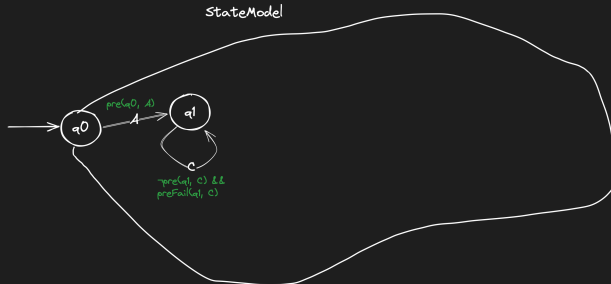
Testing for safety

- ▶ We often want to test that *nothing bad can happen*, eg. safety properties
- ▶ We can express that in the model through *Polarity of Actions*:
 - ▶ A *Positive* action is valid w.r.t to the state and is expected to *succeed* when run
 - ▶ A *Negative* action is invalid in current state and is expected to *fail*

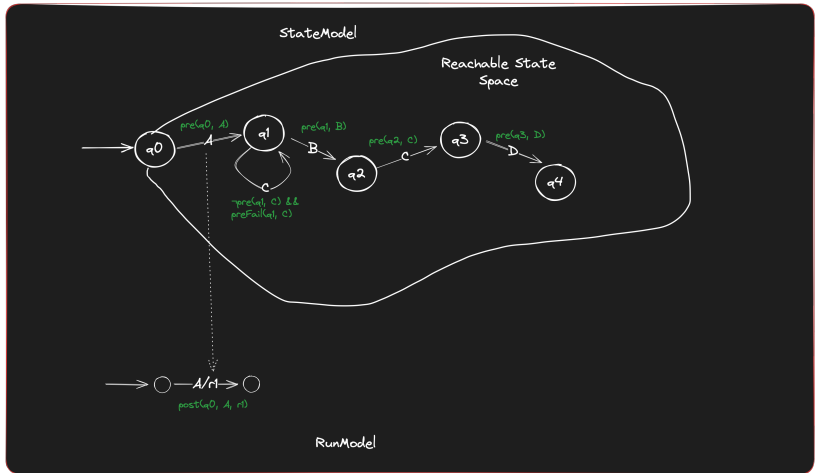
Testing for safety



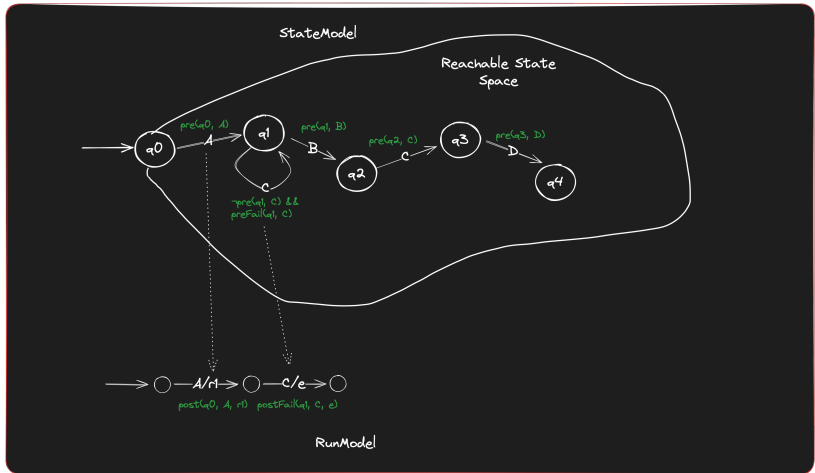
Testing for safety



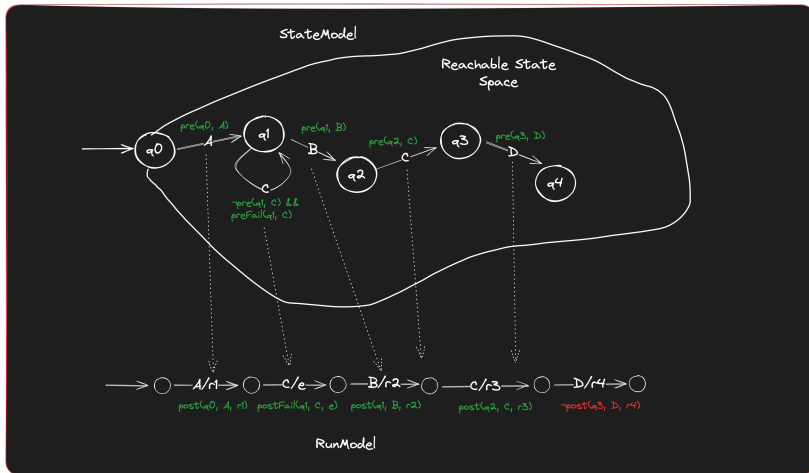
Testing for safety



Testing for safety



Testing for safety



Conclusion

Reflecting on practical use

- ▶ Tests execution does not catch much bugs beyond some regressions

Reflecting on practical use

- ▶ Tests execution does not catch much bugs beyond some regressions
- ▶ *But* test failures in development often pinpoints blind spots and misunderstandings

Reflecting on practical use

- ▶ Tests execution does not catch much bugs beyond some regressions
- ▶ *But* test failures in development often pinpoints blind spots and misunderstandings
- ▶ Main benefit is to help us *clarify* and *formalise* our thoughts on the protocols

Reflecting on practical use

- ▶ Tests execution does not catch much bugs beyond some regressions
- ▶ *But* test failures in development often pinpoints blind spots and misunderstandings
- ▶ Main benefit is to help us *clarify* and *formalise* our thoughts on the protocols
- ▶ It requires non-negligible investment to build and maintain

Tips & Tricks

- ▶ Model does not have to be *unique*

Tips & Tricks

- ▶ Model does not have to be *unique*
 - ▶ $\Rightarrow$ *Tailor models to specific problems*

Tips & Tricks

- ▶ Model does not have to be *unique*
 - ▶ $\Rightarrow$ *Tailor models to specific problems*
- ▶ Model is code hence can be buggy

Tips & Tricks

- ▶ Model does not have to be *unique*
 - ▶ $\Rightarrow$ *Tailor models to specific problems*
- ▶ Model is code hence can be buggy
 - ▶ $\Rightarrow$ *Test-drive your generators, shrinkers, doubles...*

Tips & Tricks

- ▶ Model does not have to be *unique*
 - ▶ $\Rightarrow$ *Tailor models to specific problems*
- ▶ Model is code hence can be buggy
 - ▶ $\Rightarrow$ *Test-drive your generators, shrinkers, doubles...*
- ▶ It's easy to delude yourself

Tips & Tricks

- ▶ Model does not have to be *unique*
 - ▶ $\Rightarrow$ *Tailor models to specific problems*
- ▶ Model is code hence can be buggy
 - ▶ $\Rightarrow$ *Test-drive your generators, shrinkers, doubles...*
- ▶ It's easy to delude yourself
 - ▶ $\Rightarrow$ *See it fail!*

Tips & Tricks

- ▶ Model does not have to be *unique*
 - ▶ $\Rightarrow$ *Tailor models to specific problems*
- ▶ Model is code hence can be buggy
 - ▶ $\Rightarrow$ *Test-drive your generators, shrinkers, doubles...*
- ▶ It's easy to delude yourself
 - ▶ $\Rightarrow$ *See it fail!*
- ▶ StateModel works with symbolic values

Tips & Tricks

- ▶ Model does not have to be *unique*
 - ▶ $\Rightarrow$ *Tailor models to specific problems*
- ▶ Model is code hence can be buggy
 - ▶ $\Rightarrow$ *Test-drive your generators, shrinkers, doubles...*
- ▶ It's easy to delude yourself
 - ▶ $\Rightarrow$ *See it fail!*
- ▶ StateModel works with symbolic values
 - ▶ $\Rightarrow$ *Model expectations in postcondition*

Tips & Tricks (contd.)

- ▶ Detailed modeling leads to complex code

Tips & Tricks (contd.)

- ▶ Detailed modeling leads to complex code
 - ▶ $\Rightarrow$ *Abstract the uninteresting bits away*

Tips & Tricks (contd.)

- ▶ Detailed modeling leads to complex code
 - ▶ $\Rightarrow$ *Abstract the uninteresting bits away*
- ▶ Acting on and observing internal state is tricky

Tips & Tricks (contd.)

- ▶ Detailed modeling leads to complex code
 - ▶ $\Rightarrow$ *Abstract the uninteresting bits away*
- ▶ Acting on and observing internal state is tricky
 - ▶ $\Rightarrow$ *Make the SUT testable and observable*

Tips & Tricks (contd.)

- ▶ Detailed modeling leads to complex code
 - ▶ $\Rightarrow$ *Abstract the uninteresting bits away*
- ▶ Acting on and observing internal state is tricky
 - ▶ $\Rightarrow$ *Make the SUT testable and observable*
- ▶ IO Dependencies make tests slow and brittle

Tips & Tricks (contd.)

- ▶ Detailed modeling leads to complex code
 - ▶ $\Rightarrow$ *Abstract the uninteresting bits away*
- ▶ Acting on and observing internal state is tricky
 - ▶ $\Rightarrow$ *Make the SUT testable and observable*
- ▶ IO Dependencies make tests slow and brittle
 - ▶ $\Rightarrow$ *Abstract dependencies behind simulatable interfaces*

Tips & Tricks (contd.)

- ▶ Detailed modeling leads to complex code
 - ▶ $\Rightarrow$ *Abstract the uninteresting bits away*
- ▶ Acting on and observing internal state is tricky
 - ▶ $\Rightarrow$ *Make the SUT testable and observable*
- ▶ IO Dependencies make tests slow and brittle
 - ▶ $\Rightarrow$ *Abstract dependencies behind simulatable interfaces*
- ▶ Triggering environment failures

Tips & Tricks (contd.)

- ▶ Detailed modeling leads to complex code
 - ▶ $\Rightarrow$ *Abstract the uninteresting bits away*
- ▶ Acting on and observing internal state is tricky
 - ▶ $\Rightarrow$ *Make the SUT testable and observable*
- ▶ IO Dependencies make tests slow and brittle
 - ▶ $\Rightarrow$ *Abstract dependencies behind simulatable interfaces*
- ▶ Triggering environment failures
 - ▶ $\Rightarrow$ *Simulate runtime environment for fault injections*

Tips & Tricks (end)

Model-Based Testing work hand-in-hand with Test-Driven Development

What's next?

Development on quickcheck-dynamic is fairly active, with plans for:

- ▶ Parallel testing of properties for “race conditions” detection (à la quickcheck-state-machine)

What's next?

Development on quickcheck-dynamic is fairly active, with plans for:

- ▶ Parallel testing of properties for “race conditions” detection (à la quickcheck-state-machine)
- ▶ Improved shrinking

What's next?

Development on quickcheck-dynamic is fairly active, with plans for:

- ▶ Parallel testing of properties for “race conditions” detection (à la quickcheck-state-machine)
- ▶ Improved shrinking
- ▶ More documentation, examples, use cases. . .

Takeaway

A plan is useless, planning is everything
Gal. von Moltke

Takeaway

*A model is useless, modeling is everything
quickcheck-dynamic*

Questions?

